

Structures Dynamiques Séquentielles : Les Listes Chainées

I. Les Listes chaînées simples

1. Rappel

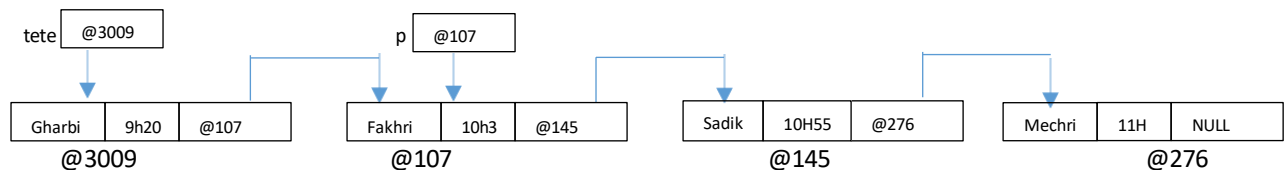
Une liste est un ensemble d'objets de même type. Chaque élément contient deux principales informations :

- un contenu utilisé à l'utilisateur (nom, prénom, note, prix, tél,...)
- L'adresse de l'élément suivant de la liste ou la valeur NULL s'il n'y a pas d'élément suivant.

La liste est accessible grâce à l'adresse de son premier élément, stockée dans une variable appelée tête.

Exemple :

Voici un exemple de la représentation en mémoire de la liste des patients selon leur ordre d'arrivée dans une salle d'attente chez le médecin.



2. Implémentation et utilisation de liste chaînée en C

En algorithmique, la déclaration de variables de type liste chaînée simple peut se faire de plusieurs manières, en voici deux :

Algorithmique	Implémentation en C
Première manière : déclaration d'un seul type	
TYPE Identif_type = Enregistrement Valur : type_valeur Suivant : pointeur sur Fin Identif_type Exemple : TYPE Elemliste = Enregistrement Score : entier Suivant : pointeur sur ElemListe Fin ElemListe Var Tête, P : Pointeur sur ElemListe	<pre>typedef struct { type_valeur valeur; struct ident_type *suivant; }</pre> Exemple <pre>typedef struct { int score; struct LesScores *suivant; } LesScores; LesScores *tete, *P;</pre>
Deuxième manière : déclaration de deux types	
TYPE Type_pointeur = Pointeur sur type_liste type_liste = Enregistrement Valur : type_valeur Suivant : Type_pointeur Fin type_liste Exemple : TYPE Pliste = pointeur sur ElemListe Elemliste = Enregistrement Score : entier Suivant : Pliste Fin ElemListe Var Tête, P : pliste	<pre>typedef struct type_liste* type_pointeur; typedef struct { type_valeur valeur; type_pointeur suivant; } type_liste;</pre> Exemple <pre>typedef struct LesScores * Pliste; typedef struct { int score; Pliste suivant; } LesScores; Pliste tete, P;</pre>

Reprenons l'exemple précédent, l'implémentation en C de la liste chaînée qui représente les patients d'un médecin. Chaque patient est identifié par son nom et l'heure de son arrivée. La structure qui permet de représenter les éléments de cette liste est la suivante :

```
typedef struct patient patient ; // la structure patient s'appelle désormais patient tout court
struct patient {
    char nom [15];
    char heure [5];
    patient *suivant ;
} ;

patient *tete, *p;
```

L'accès aux différents champs des élément de la liste peut se faire de deux manières :

a. Notation proche de l'algorithmique :

Lecture du champ nom : `Scanf("%s", (*p).nom);` // la parenthèse est obligatoire pour tous les champs

Lecture du champ heure : `Scanf("%s", (*p).heure);`

Affecter null au champ suivant : `(*p).suivant = NULL;`

Avancer dans la liste :

`p = tete;`

`p = (*p).suivant ;`

b. Deuxième Notation

Lecture du champ nom : `scanf("%s", p->nom);`

Lecture du champ heure : `scanf("%s", p->heure);`

Affecter null au champ suivant : `p->suivant = NULL;`

Avancer dans la liste :

`p = tete;`

`p = p->suivant ;`

II.3 Traitements de base sur les listes chaînées simples

a) Créer une liste chaînée d'entiers contenant 0 ou plusieurs éléments avec des ajouts en tête

L'ajout des éléments s'arrête lorsque l'utilisateur saisi un entier ≤ 0

Liste RemplirListe () /* ajout en tête de liste

```
{
    Liste T, N;
    int score ;
    T = NULL ; // au cas où aucun élément n'est ajouté à la liste
    printf ("\n donnez le score, 0 pour terminer ") ;
    scanf ("%d", &score);
    while ( score > 0) {
        N = malloc(sizeof(element));
        N->val = score;
        N->suivant = T;
        T = N;
        printf ("\n donnez le score, 0 pour terminer ");
        scanf ("%d", &score) ;
    }
    return( T);
}
```

```
}
```

b) Créer une liste chaînée d'entiers contenant 0 ou plusieurs éléments avec des ajouts en queue

L'ajout des éléments s'arrête lorsque l'utilisateur saisi un entier ≤ 0

Liste RemplirListeQ () /* ajout en queue de liste

```
{
    Liste T, N, Q;
    int score ;
    T = NULL ; // au cas où aucun élément n'est ajouté à la liste
    printf ("\n donnez le score, 0 pour terminer ") ;
    scanf ("%d", &score) ;
    if (score > 0) {
        T = malloc (sizeof(element));
        T->val = score;
        T->suivant = NULL;
        Q = T;
        printf ("\n donnez le score, 0 pour terminer ");
        scanf ("%d", &score) ;
        while ( score > 0) {
            N = malloc (sizeof(element));
            N->val = score;
            Q->suivant = N;
            Q = N;
            printf ("\n donnez le score, 0 pour terminer ");
            scanf ("%d", &score) ;
        }
        Q->suivant = NULL ;
    }
    return( T);
}
```

c) Ajouter un élément au début de la liste (Ajout en tête)

Fonction ajout_tête	Procédure ajout_tête
<pre>Liste AjoutTete (Liste T) { Liste N; N = malloc (sizeof(element)); printf ("\n saisir un entier :"); scanf ("%d", & N->val); N->suivant = T; T = N; return (T); }</pre>	<pre>void AjoutTete (Liste *T) { Liste N; N = malloc (sizeof(element)); printf ("\n saisir un entier :"); scanf ("%d", & N->val); N->suivant = *T; *T = N; }</pre>

d) Ajouter un élément à la fin d'une liste non vide (ajout en queue)

```

void ajoutQueue ( Liste T ) {
    Liste N, P;
    if (T != NULL) {
        P = T;
        While (P->suivant !=NULL)
            P = P->suivant;
        N = malloc (sizeof(element));
        printf ("\n donnez un entier ");
        scanf ("%d", &N->val);
        N->suivant = NULL;
        P->suivant = N;
    }
}

```

e) Déterminer la longueur d'une liste chaînée simple.

```

int detLongueur ( Liste T ) {
    int N ;
    Liste P ;
    n = 0;
    P = T ;
    while ( P !=NULL ) {
        N = N+1;
        P = P->suivant;
    }
    retourne ( N );
}

```

f) Afficher les valeurs des éléments d'une liste chaînée dans l'ordre normal

```

void affich ( Liste T ) {
    Liste P;
    P = T;
    while ( P !=NULL) {
        printf("\t %d", P->val);
        P = P->suivant;
    }
}

```

g) Afficher les valeurs des éléments d'une liste chaînée dans l'ordre inverse (récursive)

```

void affichInverse ( Liste T ) {
    if (T !=NULL) {
        affichInverse (T->suivant);
        printf("\t %d", T->val);
    }
}

```

h) Recherche d'un élément dans une liste chaînée

1ere version : avec une variable booléenne	2eme version : sans une variable booléenne
<pre> int recherche (Liste T, int X) { int test ; Liste P; test = 0; P = T ; while (P !=NULL) && (test == 0) { if (P->val == X) test = 1; P = P->suivant; } return (test); } </pre>	<pre> int recherche (Liste T, int X) { Liste P; P = T ; while (P !=NULL) && (P->val !=X) P = P->suivant; return (P !=NULL); } </pre>

i) Supprimer un élément à la tête de la liste

Fonction	Procédure (passage par adresse)
<pre> Liste supprimTete (Liste T) { Liste P; if (T !=NULL) { P = T; T=T->suivant; free (P); } return (T); } </pre>	<pre> void supprimTete (Liste * T) { Liste P; if (*T !=NULL) { P = *T; *T = (*T) -> suivant; free (P); } } </pre>

j) Supprimer le dernier élément de la liste

```

Liste supprimQueue ( Liste T ) {
    Liste P, R;
    if ( T !=NULL ) {
        P = T;
        while (P->suivant !=NULL) {
            R = P;
            P = P->suivant;
        }
        if (P == T) { // le dernier élément = premier élément
            T=NULL;
            free ( P);
        }
        else {
            R->suivant = NULL;
            free (P);
        }
    }
    return ( T );
}

```

}